

Sprint 7 · ML Supervisado

Regresión Lineal, Polinómica y Regularización

En este sprint aprenderás a construir modelos de regresión desde los fundamentos matemáticos hasta la implementación práctica en Python con scikit-learn. Exploraremos cómo predecir variables continuas, cómo evitar el sobreajuste y cómo aplicar técnicas de regularización para obtener modelos robustos y generalizables.

SPRINT 7 | ML SUPERVISADO | SCIKIT-LEARN

La Idea de la Regresión Lineal

La regresión lineal es el modelo predictivo más fundamental del aprendizaje automático supervisado. Su objetivo es encontrar la **línea recta** (o hiperplano, en múltiples dimensiones) que mejor se ajusta a los datos, de modo que podamos predecir una variable continua **y** a partir de una o más variables de entrada **X**.

Intuición clave

Dado un conjunto de puntos en un plano, queremos trazar la línea que minimiza la distancia (error) entre las predicciones y los valores reales.

La ecuación fundamental

El modelo toma la forma:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

Donde β_0 es el intercepto, β_i son los coeficientes y ε es el error.

Interpretación de los Coeficientes

Los coeficientes **β** son el corazón del modelo: nos dicen cuánto cambia la variable objetivo por cada unidad de cambio en una variable predictora, *manteniendo el resto constantes*. Esta interpretación es lo que hace la regresión lineal tan valiosa más allá de la predicción.

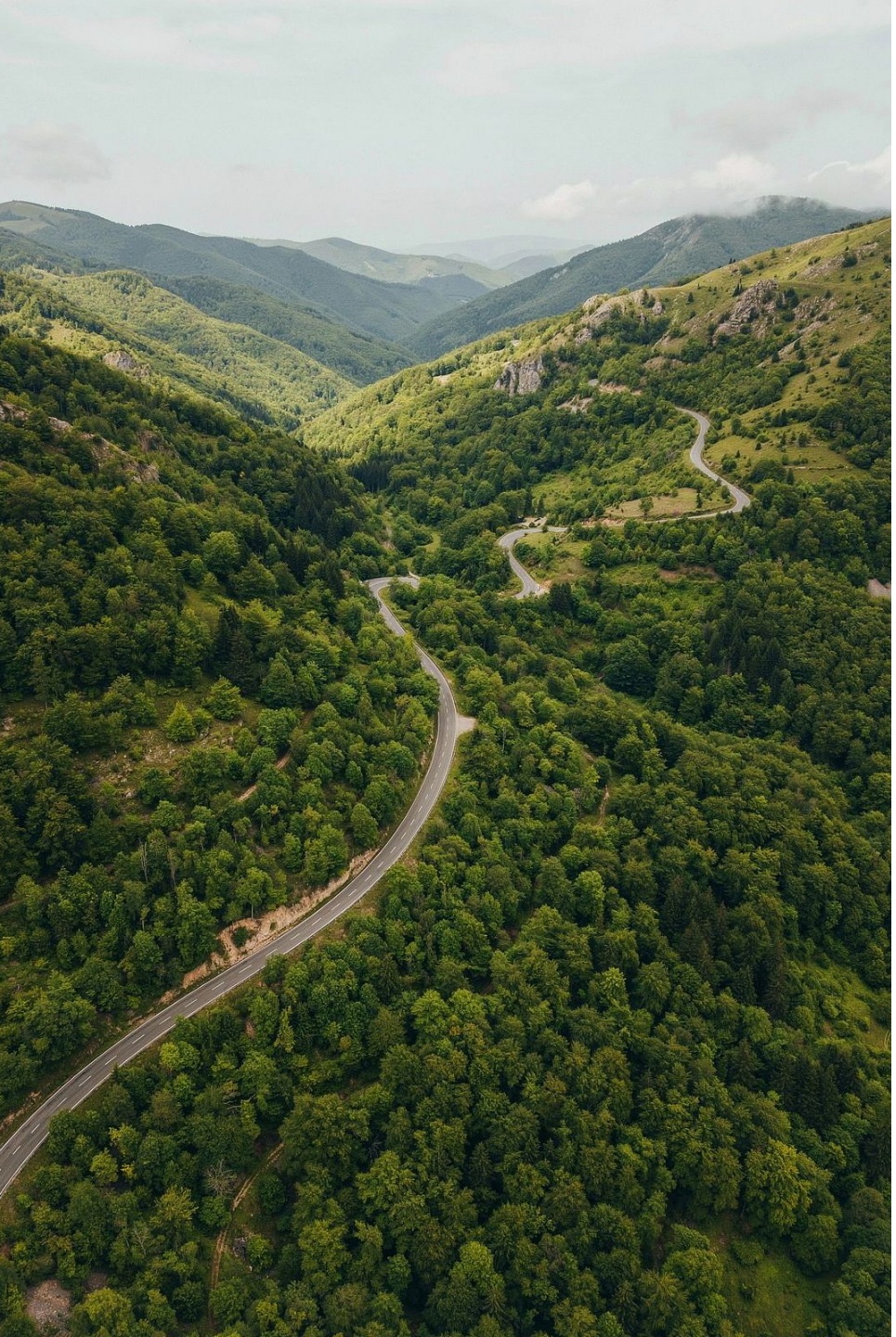
Coeficiente $\beta_1 = 45$

Si el modelo predice el precio de un alojamiento y **$\beta_1 = 45$** para la variable "número de habitaciones", esto significa: *cada habitación adicional aumenta el precio en 45 € de media*, con el resto de variables fijas.

Signos e importancia

- **β positivo**: la variable predictora tiene relación directa con y.
- **β negativo**: relación inversa. Más X implica menos y.
- **β ≈ 0**: esa variable apenas contribuye al modelo.
- **p-valor**: indica si el coeficiente es estadísticamente significativo.

Con variables en distintas escalas, escala los datos antes de comparar la magnitud de los coeficientes.



Regresión Polinómica

Cuando la relación entre variables no es lineal, podemos **elevar las variables de entrada a potencias superiores** para capturar curvas y patrones más complejos. Esto se llama regresión polinómica, y técnicamente sigue siendo un modelo lineal en sus parámetros.

Sobreajuste: El Gran Enemigo del ML

Un modelo que aprende **demasiado bien** los datos de entrenamiento —incluyendo el ruido— falla a la hora de generalizar a datos nuevos. Este fenómeno se llama **sobreajuste (overfitting)** y es uno de los problemas más frecuentes en aprendizaje automático.

Infraajuste

Modelo demasiado simple, alto error en train y test.

Sobreajuste

Modelo complejo, bajo error en train y alto en test.



Ajuste óptimo

Modelo equilibrado, bajo error en train y test.

La clave está en el **equilibrio sesgo-varianza**: modelos simples tienen alto sesgo (underfitting), modelos complejos tienen alta varianza (overfitting).



Regularización: Controlando la Complejidad

La regularización es una técnica que **añade una penalización a la función de coste** proporcional a la magnitud de los coeficientes. De este modo, el modelo se ve "forzado" a ser más sencillo, reduciendo el sobreajuste sin necesidad de reducir el número de variables manualmente.

$$J_{reg}(\beta) = \underbrace{\|X\beta - y\|^2}_{\text{Error de ajuste}} + \underbrace{\lambda \cdot \Omega(\beta)}_{\text{Penalización}}$$

El parámetro **λ (lambda)** controla el equilibrio: λ=0 equivale a regresión sin regularización; λ muy grande produce coeficientes cercanos a cero.

Ridge vs. Lasso: Dos Enfoques

Las dos técnicas de regularización más usadas difieren en el tipo de penalización que aplican sobre los coeficientes, lo que produce comportamientos distintos y hace que cada una sea más adecuada según el contexto.

Ridge (L2)

$$\lambda \sum_j \beta_j^2$$

Penaliza el **cuadrado** de los coeficientes. Los reduce hacia cero pero **nunca los elimina del todo**. Ideal cuando todas las variables son potencialmente relevantes. Especialmente útil ante multicolinealidad.

Lasso (L1)

$$\lambda \sum_j |\beta_j|$$

Penaliza el **valor absoluto** de los coeficientes. Puede llevarlos exactamente a cero, realizando una **selección automática de variables**. Produce modelos más interpretables y dispersos (sparse).

Elastic Net combina ambas penalizaciones (L1 + L2) para obtener lo mejor de cada enfoque.

Caso Práctico: Predicción del Precio por Noche

Aplicamos todo lo aprendido a un problema real: **predecir el precio por noche de un alojamiento** a partir de sus características. Este tipo de modelo es el núcleo de los sistemas de fijación de precios dinámicos usados en plataformas como Airbnb.

CASO REAL | REGRESIÓN | AIRBNB-STYLE



Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S07_01_regresion_lineal_polinomica — 40 min

Regresión lineal y polinómica

sprints/S07/notebooks/S07_01_regresion_lineal_polinomica.ipynb

CUADERNO

S07_02_ridge_lasso — 35 min

Ridge y Lasso

sprints/S07/notebooks/S07_02_ridge_lasso.ipynb